

PROGRAMAÇÃO MODULAR

TESTES UNITÁRIOS: UMA INTRODUÇÃO

PROF. JOÃO CARAM

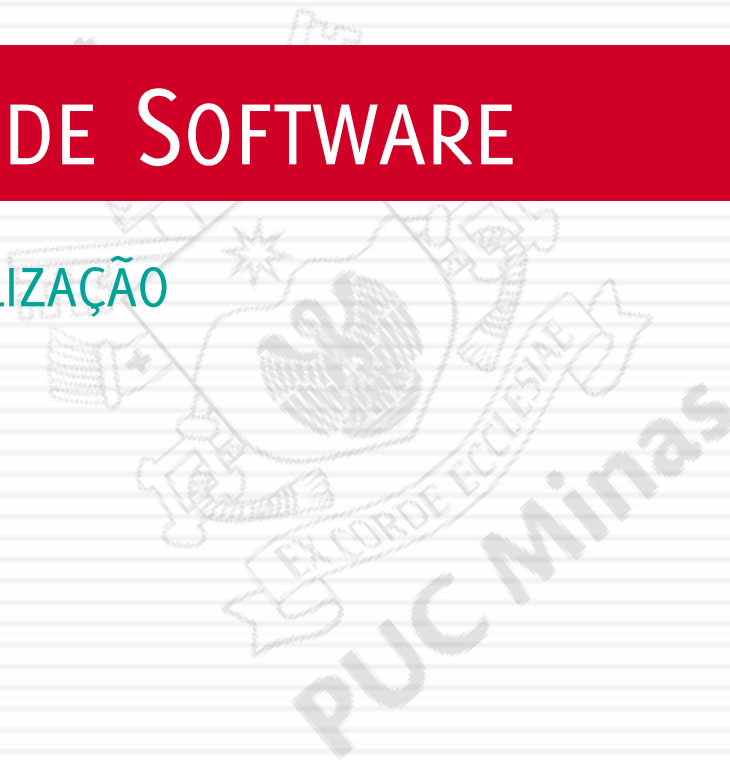
PUC MINAS

BACHARELADO EM ENGENHARIA DE SOFTWARE

2

TESTES DE SOFTWARE

CONTEXTUALIZAÇÃO



SOFTWARE DE QUALIDADE

3

- Número e severidade dos defeitos aceitáveis.
- Entregue dentro do prazo e custo.
- Atende aos requisitos/expectativas.
- Pode ser mantido eficientemente após a implantação.

(Rios; Moreira Filho, 2013)

TESTE DE SOFTWARE!

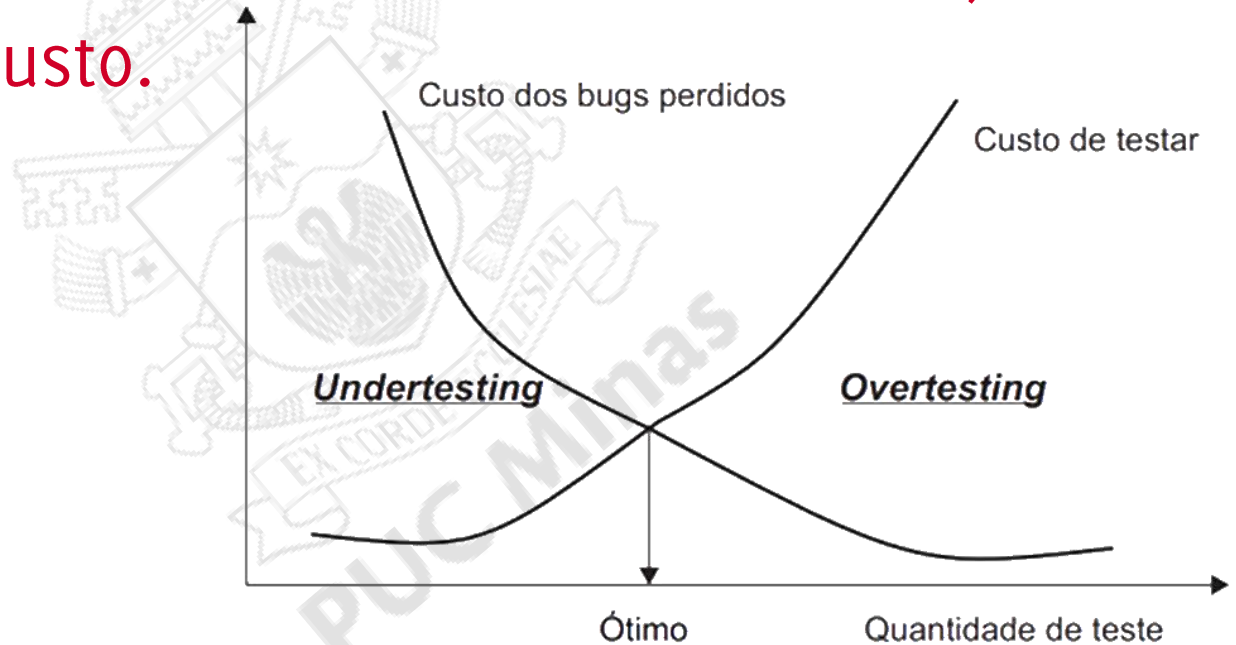
4

- Uma maneira de se garantir a qualidade!
- Mostrar que um programa faz o que é proposto e descobrir defeitos antes do uso. *(Sommerville, 2019)*
- Um conjunto de atividades que podem ser planejadas antecipadamente e conduzidas sistematicamente. *(Pressman;Maxim, 2016)*

TESTE DE SOFTWARE!

5

- Quanto mais se demora a detectar um erro, maior será o seu custo.



Adaptado de: *Beginning Java Programming* (2015) - Bart Baesens and Aimée Backiel; Capítulo 1, pg. 7

6

TESTE UNITÁRIO



TESTE UNITÁRIO

7

- Testa cada componente (classe, método, rotina, página...) individualmente.
- Objetivo principal: garantir que cada unidade, isoladamente, funciona de acordo com sua especificação.

TEORIA DO TESTE UNITÁRIO

8

- *“Para cada trecho de código operacional – uma classe ou um método –, o código operacional deve estar pareado com um código de teste unitário.”*

“If you can’t write a test for what you are about to code, then you shouldn’t even be thinking about coding.” (George; Williams, 2003)

TESTE UNITÁRIO

9

- Chama o código operacional a partir da sua interface pública, de diversas formas, e verifica os resultados.
- Não precisa ser exaustivo.
 - código de teste já agrega valor, mesmo considerando somente casos centrais.

TESTE UNITÁRIO COM XUNIT E JUNIT

10

- xUnit: nome genérico para uma coleção de plataformas de testes unitários.
 - SUnit (Kent Beck, 1998).
- A primeira letra costuma indicar a plataforma.
 - JUnit, RUnit, CUnit, CppUnit, ShUnit, JSUnit...
 - xUnit.net, minitest, Pytest...

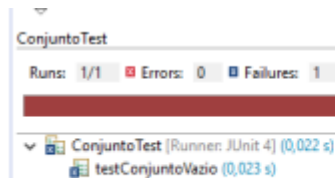
TESTE UNITÁRIO

11

Feedback vermelho: erro.

```
50  
51 @Test  
52 public void criarConjuntoVazio(){  
53     conj = new Conjunto(tamanho:0);  
54     assertEquals(0, conj.tamMaximo());  
55 }
```

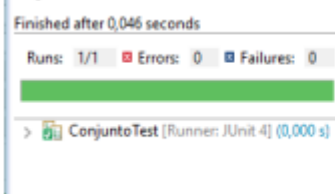
Expected [0] but was [1]



```
5 public class ConjuntoTest {  
6  
7     @Test  
8     public void testConjuntoVazio() {  
9         Conjunto c = new Conjunto();  
10        assertEquals(0, c.tamanho());  
11    }  
12  
13 }
```

Feedback verde: sucesso.

```
51 @Test  
52 public void criarConjuntoVazio(){  
53     conj = new Conjunto(tamanho:0);  
54     assertEquals(0, conj.tamMaximo());  
55 }
```



```
5 public class ConjuntoTest {  
6  
7     @Test  
8     public void testConjuntoVazio() {  
9         Conjunto c = new Conjunto();  
10        assertEquals(0, c.tamanho());  
11    }  
12 }
```

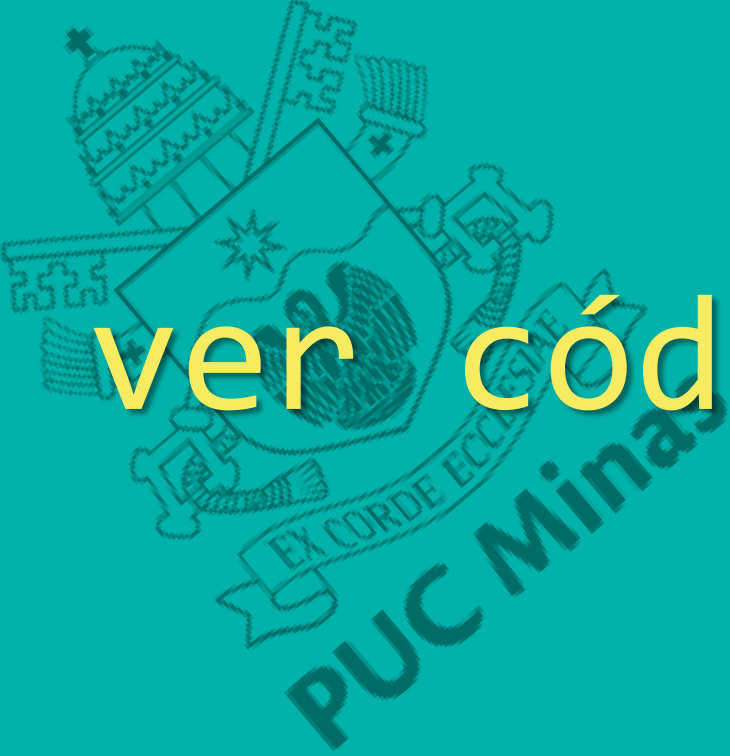
TESTE UNITÁRIO

12

▣ Padrão AAA: arrange, act, assert.

- ▣ *Arrange* (organizar): preparação, incluindo criação de objetos, inicialização de variáveis, entre outros.
- ▣ *Act* (agir): chamada/execução do método sendo testado.
- ▣ *Assert* (verificar): comparar o resultado obtido com o que era esperado.

{ Quero ver código!! }

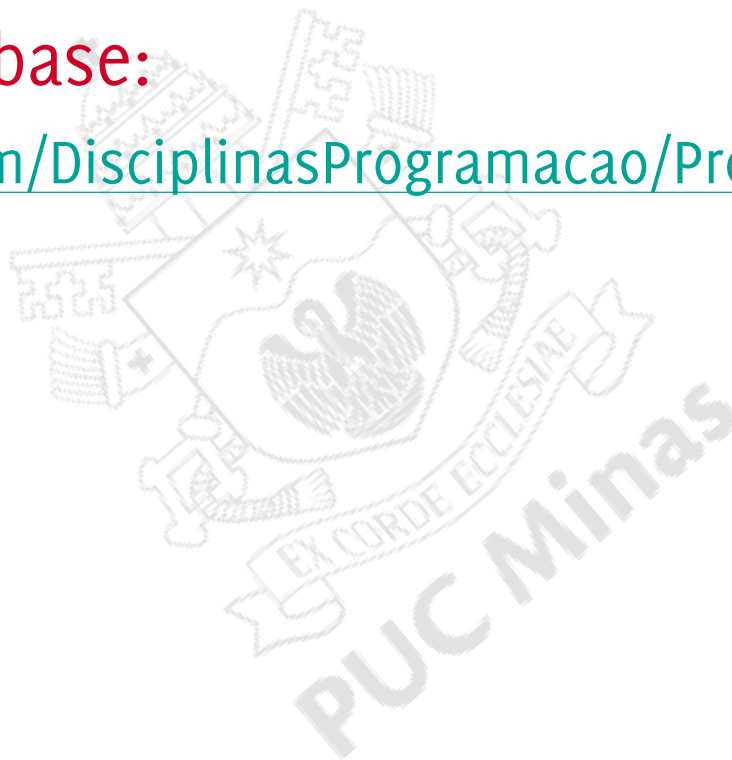
A large, faint watermark of the PUC Minas logo is visible in the background. It features a shield with a star, a banner with the text "EX CORDE ECCLESIAE", and the text "PUC Minas" below it.

FORK + CLONE

14

▣ Repositório-base:

https://github.com/DisциплиnasProgramacao/ProgMod_TestesUnitarios



TESTANDO A CLASSE HORA

15

- Uma hora armazena valores inválidos?
- O incremento de minutos gera uma hora correta?
- A comparação entre horas funciona corretamente?
- A formatação está correta?

DEPURAÇÃO / *DEBUGGING*

16

- Processo de depurar o código para remover os erros de programação (*bugs*).
 - Detectar, localizar e solucionar o erro.

COMO TESTAR / CASOS DE TESTE

17

- Básicos:
 - entradas simples e óbvias que devem funcionar. São adicionados primeiro.
- Borda/fronteira:
 - casos simples que representam condições extremas – lista vazia, lista cheia, valores-limite

COMO TESTAR / CASOS DE TESTE

18

- Avançados:
 - casos de teste mais complexos.
 - Normalmente feitos posteriormente, quando um maior conhecimento do problema leva a identificar potenciais casos estranhos.

19

JUNIT - ASSERÇÕES



JUNIT E ASSERTS

20

Assert	Objetivo
<code>assertTrue(teste)</code>	falha se teste booleano é false .
<code>assertFalse(teste)</code>	falha se teste booleano é true .
<code>assertEquals(esperado, teste real)</code>	falha se valores não são iguais.
<code>assertNull(teste)</code>	falha se valor não for null .
<code>assertNotNull(teste)</code>	falha se valor for null .

JUNIT E ASSERTS

21

Assert	Objetivo
<code>assertThrows(classe esperada, execução)</code>	falha se a exceção não é lançada.
<code>assertSame(esperado, teste real)</code>	falha se valores não são os mesmos (por meio de <code>==</code>).
<code>assertNotSame(esperado, teste real)</code>	falha se valores são os mesmos (por meio de <code>==</code>).
<code>fail ()</code>	o teste interrompe sua execução e falha.

STRING DE INFORMAÇÃO

22

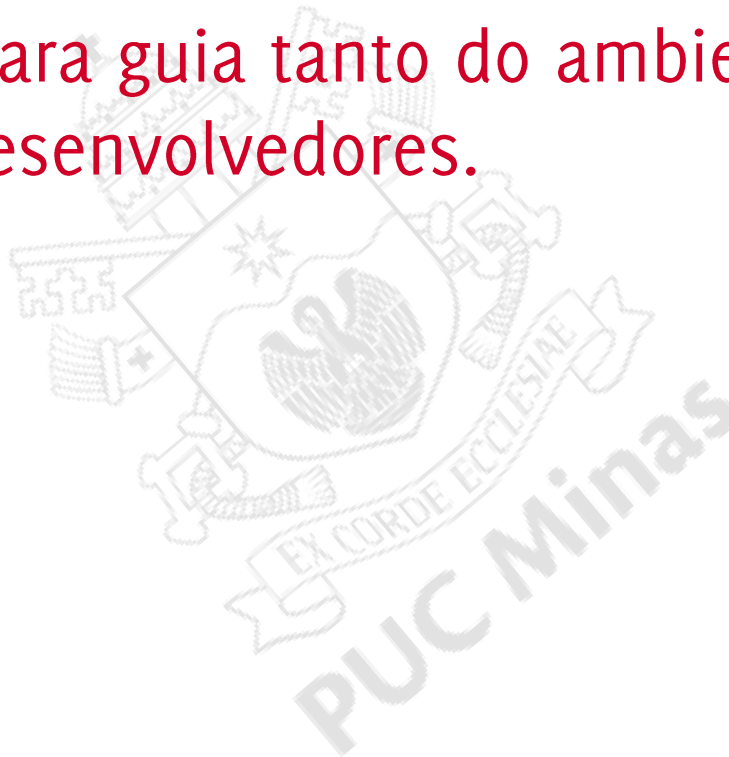
- ▣ Parâmetro adicional, exibido caso o teste falhe:

```
Data data1 = new Data(10,10,2025);  
Data data2 = data1.somarDias(22);  
assertEquals("01/11/2025", data2.dataFormatada(),  
             "Teste de data anterior a outra");
```

JUNIT, ANOTAÇÕES E INFORMAÇÕES

23

- Anotações para guia tanto do ambiente de testes como dos desenvolvedores.



@DISPLAYNAME

24

- Define um nome/descrição para a classe ou o método de teste.

```
@Test
```

```
@DisplayName("Soma de dias na data")
```

```
public void somaDiasCorretamente() {
```

```
    ...
```

```
}
```


INICIALIZAÇÃO E FINALIZAÇÃO

25

- Métodos executados antes ou depois de cada caso de teste:

@BeforeEach

```
void setUp() throws Exception {  
}
```

@AfterEach

```
void tearDown() throws Exception {  
}
```

INICIALIZAÇÃO E FINALIZAÇÃO

26

- Métodos executados uma única vez, antes ou depois da execução de toda a classe de testes:

@BeforeAll

```
static void setUpBefore() throws Exception {  
}
```

@AfterAll

```
void tearDownAfter() throws Exception {  
}
```

@TestMethodOrder

27

- Define a modo de ordem de execução dos testes:
 - MethodOrderer.MethodName.class
 - OrderAnnotation.class
 - @Order(*n*)
 - MethodOrderer.Random.class

OBRIGADO.

DÚVIDAS?